

02 Architecture Analysis — 6 Interfaces

#1 Battery → Inverter

CAN Bus

Safety-critical, bus-off = silent failure

#2 Inverter → IoT

Modbus RTU

PV Production + Battery Power — most critical

PV Production + Battery Power feed House Consumption calc

#3 Power Meter → IoT

Modbus RTU

Grid Power — feeds House Consumption calculate

#4 Power Meter → IoT

Modbus TCP

Via Switch — stale TCP

#5 IoT → Cloud

MQTT/Ethernet

Buffer + reconnect — data loss during outage

#6 CT Clamps → Power Meter

Physical

Hardware only — backwards clamp is silent

KEY INSIGHT: IoT is the hub — it sits on every interface except CAN. Any failure there = silent wrong data on dashboard.

L1 — Unit Testing | L2 — Integration Testing

L1 · Unit / Component

- Modbus register scaling factor logic
- CRC validation and error handling
- House Consumption formula — all 4 quadrants
- MQTT payload JSON schema validation
- Buffer overflow
- Sign convention — import vs export for grid

No real hardware needed · Simulators only

L2 · Integration Testing

- IoT ↔ Inverter — Modbus RTU session & polling
- IoT ↔ Power Meter — Modbus RTU direct serial
- IoT ↔ Power Meter — Modbus TCP via Switch
- Battery ↔ Inverter — CAN bus, bus-off injection
- Both RTU ports polled simultaneously — timing gap
- TCP failover to RTU when Switch goes down
- IoT ↔ Mock Cloud — MQTT, retry, backoff logic

Real protocols · Simulators replace real HW · Test each interface alone, then combined

Test interfaces in isolation first → fault isolation. Then test combined → catch emergent failures invisible to either alone.

L3 — System / HiL Testing | L4 — Acceptance / Regression

L3 · System / HiL Testing

- CT Clamp reference load — programmable AC source
- Golden dataset — inject known → assert at cloud
- 24-hour soak test — drift, memory leak, gaps
- Network outage — buffer fill and reconnect
- Power supply interruption — clean restart
- CAN bus-off — stale battery data detection
- Buffer overflow — alert at 80%, loss reporting

**Real IoT device, real Power Meter, real Switch, programmable AC source
— cannot be skipped**

L4 · Acceptance / Regression

- All 4 measurements correct on dashboard
- End-to-end latency within agreed SLA (<30s)
- No time-series gaps during normal operation
- Correct values after device reboot
- Correct behaviour after network reconnect
- Regression gate — automated on every release

Business-facing · Validates the use case · Automated release gate · Robot Framework

L3 is the most critical level — House Consumption is derived and CT Clamp errors are physical. Software alone cannot catch them.

04 Test Hardware Integration Points

SIMULATOR

Interface 1 — CAN

python-can simulator

Full control over frames, bus-off injection, value scripting.
CANoe for lab debugging only.

SIMULATOR

Interface 2 — Modbus RTU (Inverter)

pymodbus slave

Inject exact register values, wrong CRC, timeouts. Real inverter cannot do this on demand.

MIXED

Interface 3 — Modbus RTU (Power Meter)

pymodbus + real HW

Simulator for logic.
Real Power Meter + AC source for sign convention physical validation.

MIXED

Interface 4 — Modbus TCP (Switch)

pymodbus TCP + real Switch

Keep real Switch — failover, IP conflict, and VLAN tests need real network behaviour.

SIMULATOR

Interface 5 — IoT to Cloud

Mock cloud + tc netem

Mock cloud for payload/retry testing. tc netem injects latency, packet loss, outage.

REAL HW

Interface 6 — CT Clamps

Programmable AC source

MANDATORY — backwards clamp, wrong phase, drift — invisible to software. Physical only.

Rule: Real hardware only where software simulation cannot replicate the failure mode.

06 Risks & Mitigations — All Silent Failures

HIGH

CT Clamp misinstallation

Risk: Wrong phase, backwards, or not closed → wrong Grid Power → wrong House Consumption. Entire software stack works normally.

Test: Programmable AC source injects known 3500W. Assert Power Meter shows 3500W $\pm 1\%$. Check all 3 phases, check sign.

HIGH

Timing gap between Modbus reads

Risk: IoT polls Inverter and Power Meter sequentially. House Consumption calculated from two different moments in time.

Test: Both simulators simultaneously, rapidly changing values. Measure timestamp delta. Assert calculation accuracy within bounds.

HIGH

Data loss during network outage

Risk: Internet drops, IoT has no local buffer. 2 hours of measurements lost silently. No alert to operations team.

Test: tc netem cuts Internet for 5min / 1hr / 8hr. Assert all data buffered, uploaded on reconnect, no gaps in dashboard.

MED

Buffer overflow — silent discard

Risk: Extended outage fills local buffer. New measurements silently discarded. No notification. Data permanently lost.

Test: Outage longer than buffer capacity. Assert: 80% alert fires, overflow is logged, loss period marked on dashboard.

MED

CAN bus-off — stale battery data

Risk: Battery CAN node enters bus-off. IoT keeps receiving last known value from Inverter. Dashboard looks live — data is stale.

Test: python-can injects bus-off condition. Assert IoT raises fault flag. Assert dashboard shows error — not last known value.

05 Tools & Frameworks

pytest

L1 / L2

Primary test runner for IoT firmware unit tests and integration tests. pytest-asyncio for async Modbus polling.

pymodbus

L1 / L2

Modbus RTU + TCP client and slave simulator. Scriptable register maps, CRC injection, timeout simulation.

python-can

L1 / L2

CAN bus simulator for Battery-Inverter interface. Injects bus-off, frame loss. Integrates with pytest.

Wireshark / tshark

L2

Passive Modbus TCP packet capture. Protocol conformance, timing analysis, unexpected traffic detection.

tc netem

L2 / L3

Linux network impairment. Injects latency, packet loss, bandwidth limits. Simulates Internet outage.

Robot Framework

L3 / L4

Keyword-driven energy scenario scripts. Human-readable: "Set PV To 5000W → Verify Dashboard Shows 3000W".

Grafana + InfluxDB

L3

Soak test visualisation. Shows measurement drift, time-series gaps, latency spikes over 24 hours.

Jenkins

All

CI pipeline. L1 on every commit, L2 on every PR, L3 nightly on HiL rig. Blocks release on failure.

All tools integrate into Jenkins — entire suite runs automatically. No manual test execution on release.

Key Insights

- 1** Map ALL interfaces before proposing any tests — 6 interfaces, each with distinct failure modes at different testing levels.
- 2** House Consumption is derived, never measured — errors in either source propagate silently to every calculation that depends on it.
- 3** Silent failures dominate this architecture — wrong register, wrong scaling, stale CAN data, backwards CT Clamp — all look normal.
- 4** L3 HiL testing cannot be skipped — CT Clamp physical installation errors are invisible to software regardless of test coverage.
- 5** Test in isolation first, then combined — emergent failures like timing gaps only appear when both interfaces run simultaneously.
- 6** Automate regression through CI/CD

Test Hardware Integration Strategy & Tools and Framework Across Validation Layers

L4 - End to End/ Regression

Robot Framework • Jenkins • Grafana
Backend Validation • Dashboard Checks

L3 - HIL Validation/ System testing

Canoe • Programmable Ac/DC Source
Physical Power Behavior • CT Clamp Accuracy

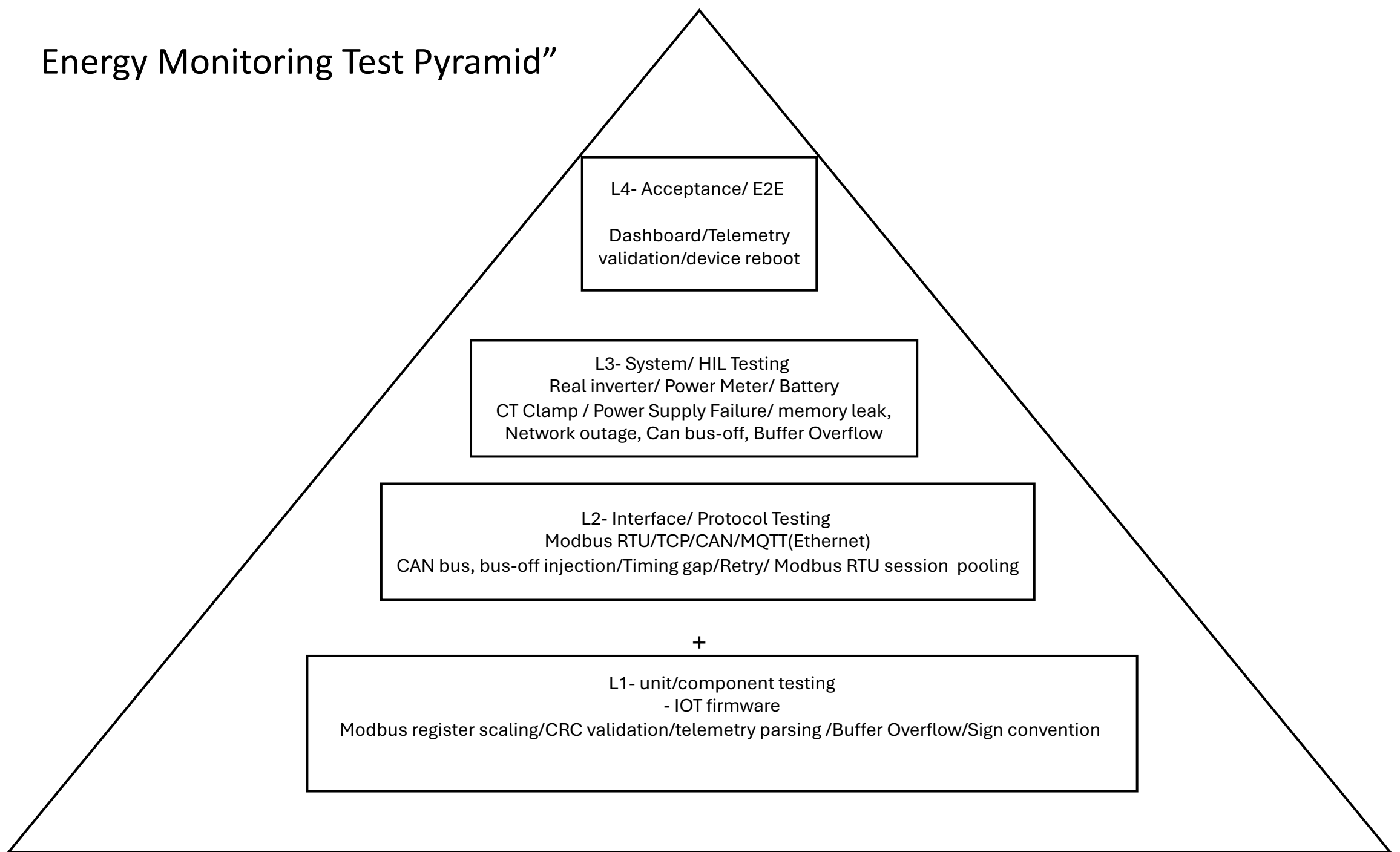
L2 - Interface / protocol Validation

Pymodbus • python-can • Canoe / Wireshark/ tc netem
Modbus RTU/TCP • CAN • Timeout • CRC • Sync Timing

L1 – Unit/Component Testing

Python • pytest • Mocked Protocol Responses
Scaling • Parsing • House Consumption Calculation

Energy Monitoring Test Pyramid”



Battery $\leftarrow$ CANoe/python-can $\leftrightarrow$ Inverter $\leftarrow$ pymodbus RTU $\rightarrow$ IoT

$\uparrow$

Programmable AC Source

Power Meter $\leftarrow$ pymodbus TCP $\leftrightarrow$ IoT

IoT $\leftarrow$ Robot Framework + Jenkins $\rightarrow$ Backend/Cloud

$\downarrow$

Grafana Monitoring